Lab 1 – Stream Ciphers: XOR, Keystream, and the Two-Time Pad

Criptografía

Facultad de Ingeniería

Departamento de Computación

Learning Objectives

- Understand and implement the model keystream ⊕ plaintext → ciphertext.
- Explain why reusing a nonce (and thus the keystream) is catastrophic.
- Perform a simple crib-dragging[1] exercise.

Overview

Students implement a deterministic keystream generator (CTR-style over SHA-256), then encrypt and decrypt messages using XOR. They will demonstrate the two-time pad attack by reusing the same nonce and show that C1 ⊕ C2 = M1 ⊕ M2.

For example, if we use these parameters:

- seed: S
- nonce: N

To encrypt the following messages

- m1: Meet me at the plaza at noon
- m2: Bring the datasets and the NDA

You create the corresponding keystream and encrypt m1 and m2, obtaining:

- C1 (hex) : d206e947c15e28177a3da31b7bb3afecb50d072c1c9309628bd20d41
- C2 (hex) : dd11e55d8613395f7e69e70e67b7fcf9ad1f5d2c52965d368dd84261deb2

Then, you can prove that C1 ⊕ C2 = M1 ⊕ M2

- C1⊕C2: 0f170c1a474d1148045444151c04531518125a004e055454060a4f20
- P1⊕P2: 0f170c1a474d1148045444151c04531518125a004e055454060a4f20

How to implement a deterministic keystream generator using SHA-256

A *keystream* does that uses a CTR-style with SHA-256 builds a byte stream (K) of any requested length by repeatedly hashing a *fixed secret* (seed), a *public per-message value* (nonce), and an ever-increasing *counter* (ctr):

[1] Crib-dragging is a classic attack against stream/CTR encryption when the same keystream (same key+nonce) was reused to encrypt two different plaintexts.

$$K_i = SHA256(seed \parallel nonce \parallel ctr_i)$$

Each digest is 32 bytes; the function concatenates these blocks until it has at least *nbytes*, then truncates.

In this case, SHA-256 behaves like a pseudorandom function when its inputs are unique and unpredictable. Because *ctr* changes (0,1,2,...) the input to SHA-256 is different for each block, so each chunk of keystream looks new and random-like.

The same (*seed*, *nonce*) pair produces the same *keystream*, so encryption is:

```
ciphertext = plaintext XOR keystream
```

Roles of the inputs.

1. *seed* (key): secret; fixes the pseudorandom function family.
2. *nonce*: public but must be unique per message under a given *seed*.
3. *ctr.to_bytes(8, 'big')*: encodes the integer counter in 8 bytes (64-bit, big-endian) to make a fixed-length, unambiguous input block.

Important remarks

This is a didactic stream generator, not a standardized stream cipher. In production is better to use AES-CTR or ChaCha20, and pair encryption with AEAD (e.g., GCM or ChaCha20-Poly1305) to get integrity.

Tasks

1) Implement functions **`keystream`** `(seed, nbytes, nonce, counter)`, **`byte-wise xor`** `(a, b)`, **`encrypt_stream`**, and **`decrypt_stream`**.
2) Verify correct round-trip encryption/decryption for several test messages.
3) Demonstrate the two-time pad: encrypt two different plaintexts with the same seed and nonce; compute C1 ⊕ C2 and confirm it equals P1 ⊕ P2.

Deliverables

- Source code implementing the functions on Alphagrader.
- All test successfully passed on Alphagrader.